



dial9

a flight recorder for Rust

Me

Russell Cohen

AWS

- AWS SDK for Rust
- `smithy-rs`
- Occasional Tokio contributor

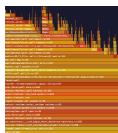
dial9: a flight recorder for Rust

Why do we need a flight recorder?

- record a large number of **events**, compactly and efficiently
- easily get them off the host for analysis

record a large number of **events**, compactly and efficiently

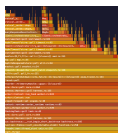
What's an "event"?



CPU stack samples

record a large number of **events**, compactly and efficiently

What's an "event"?



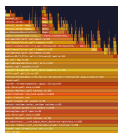
CPU stack samples



application events / **tracing**

record a large number of **events**, compactly and efficiently

What's an "event"?



CPU stack samples



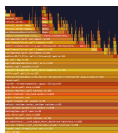
application events / **tracing**



Tokio events

record a large number of **events**, compactly and efficiently

What's an "event"?



CPU stack samples



application events / **tracing**



Tokio events

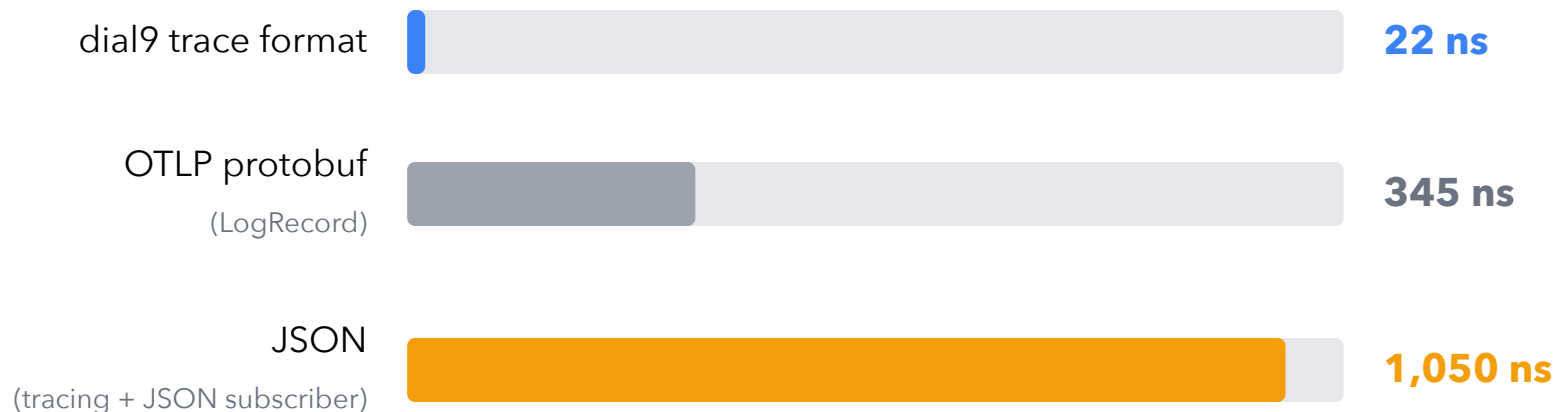


GPU telemetry

sources don't need to be part of dial9

Fast and compact event serialization

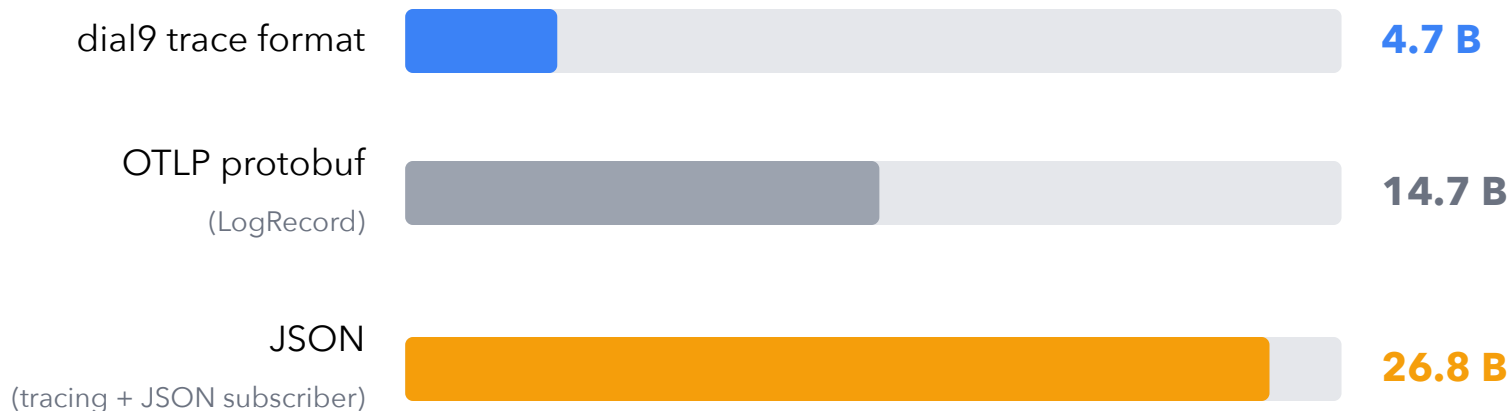
Wall-clock time to turn one event into bytes



1M-event scheduler-telemetry mix (poll start/end, park, wake, CPU samples w/ stacks) · Apple M2 · medians of 8 runs

Fast and compact event serialization

Bytes per event, after gzip level 1

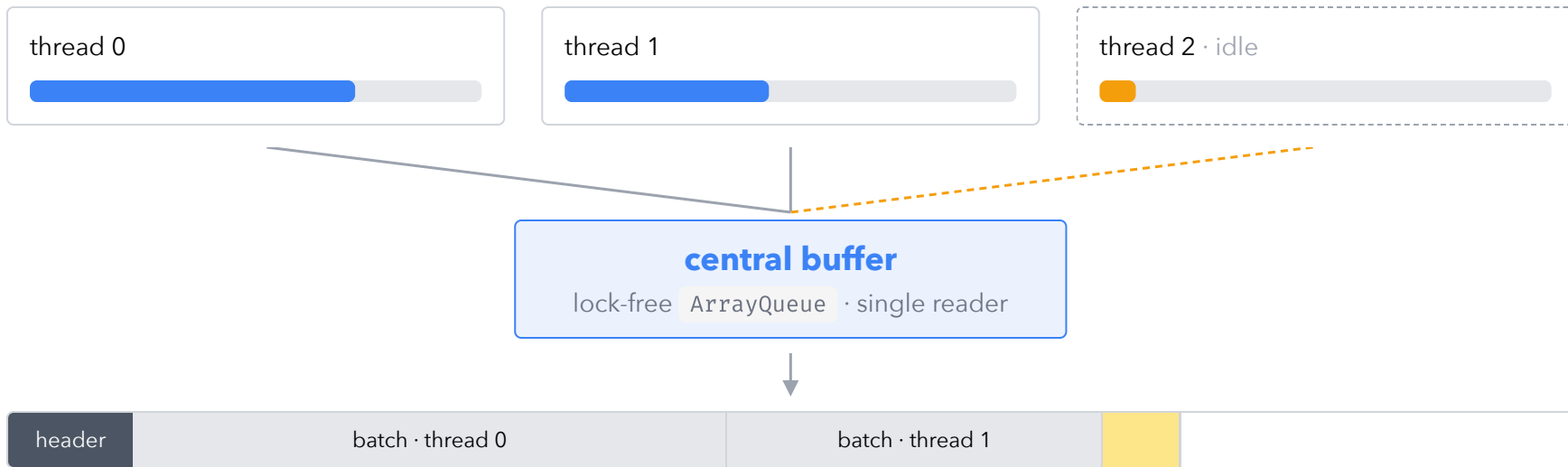


1M-event scheduler-telemetry mix (poll start/end, park, wake, CPU samples w/ stacks) · Apple M2 · medians of 8 runs

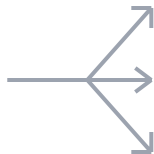
Thread-local buffers

avoid contention on the hot path

1 MB each · events encoded in place · every push checks the global epoch counter



Getting them off the host



S3

Disk

???

Where are we now?

```
cargo add dial9
```

Cargo.toml

```
[dependencies]
dial9 = { version = "0.5", features = ["tokio", "cpu-profiling"] }
```

main.rs

```
#[dial9::main(config = dial9::recorder_from_env)]
async fn main() {
    // your async code here
}
```

Demo

What's hard?

- Full Tokio instrumentation requires instrumenting every spawned task (and there is no good way to do this)
- `dial9` aims to be a "cafeteria profiler"; take what you want (more data, more runtime overhead). But this comes with complexity.

What's next?

- Better analysis especially collaborative agent analysis
- Improved platform support (Android, iOS etc.)
- Squeezing out more performance
- File an issue!

Some links

the dial9 blog:

- dial9: a flight recorder for Rust (more details about internals)
- what's new in dial9 0.5
- docs.rs